SCALE FOR PROJECT PYTHON MODULE (/PROJECTS/PYTHON-MODULE-09)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-f3439c39-98
```

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Work with the person (or the group) being evaluated to identify any potential dysfunctions in the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there may be differences in how your peers have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade them as honestly as possible. The pedagogy is valid only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student's or group's Git repository.

- Double-check that the Git repository belongs to the student or the group. Ensure that the work is for the relevant project and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases were used to fool you and make you evaluate something other than the content of the official repository.

- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed the possible scripts used to facilitate the grading.

- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository, a non-functioning program, a norm error, cheating, etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are encouraged to continue to discuss your work (even if you have not finished it) in order to identify any issues that may have caused this failure and avoid repeating the same mistake in the future.

- Remember that during the defence, no unexpected, premature, uncontrolled termination of the program; otherwise, the final grade is 0. Use the appropriate flag.
You should never need to edit any file except the configuration file, if it exists. If you want to edit a file, take the time to explain the reasons to the evaluated student and make sure both of you agree to this.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/203245/en.subject.pdf)

data_generator.tar (https://cdn.intra.42.fr/document/document/48276/data_generator.tar)

Preliminaries

Basics

- The review is done in the presence of the reviewed learner. This is how everyone progresses: by interacting with others.
    - No submitted work: 0, the review is over.
    - As soon as an exercise is non-functional, the review stops. You can look at the code o exercises, but they will not be graded.
- Check that the following files are present:
    - ex0/space_station.py
    - ex1/alien_contact.py
    - ex2/space_crew.py If any required files are missing, the peer-review stops here.
- Verify that the code runs without errors and demonstrates the required concepts.
- Test both valid and invalid data scenarios to ensure proper validation.
- Look for proper use of Pydantic features: BaseModel, Field, @model_validator.
- Ensure the learner avoided deprecated decorators (@validator) in favor of @model_validato

Yes No

Exercise 0

Exercise 0 - Space Station Data

Run the command: python3 ex0/space_station.py

Verify that:

- The SpaceStation model inherits from BaseModel
- All required fields use Field() with proper constraints
- The model accepts valid space station data
- Invalid data (like crew_size > 20) raises validation errors
- The output shows both successful creation and validation errors
- Verify the code contain a valid SpaceStation and an invalid one which raise a properly handle hardcoded output.

If the exercise doesn't work as expected, tick "No."

Yes No

Code Quality - Exercise 0

Check the code quality:

- Are the field constraints appropriate (crew_size 1-20, power_level 0-100, etc.)?
- Does the model include optional fields in 'notes'?
- Is the datetime field properly handled? What happens when you pass a string timestamp in 'last_maintenance'?
- Is the code well-structured and readable?

If the code quality is poor, tick "No."

Yes No

Exercise 1

Exercise 1 - Alien Contact Data

Run the command: python3 ex1/alien_contact.py

Verify that:

- The AlienContact model uses @model_validator for custom validation

- ContactType enum is properly defined and used
- Are each value of the enum properly defined ?
- Custom validation rules work (contact_id starts with "AC", physical contacts need verification
- message_received combine both 'Optional' typing AND a maximum length of 500 characters
- The demonstration shows both valid contacts and validation errors
- No deprecated @validator decorators are used

If the exercise doesn't work as expected, tick "No."

Yes No

Custom Validation Rules - Exercise 1

Test the custom validation rules:

- Try creating a contact with ID not starting with "AC" - does it fail?
- Try telepathic contact with < 3 witnesses - does it fail?
- Try strong signal without message - does it fail?
- Try physical contact type which is not verified - does it fail?
- Try to create a contact without specifying is.verified. Is it False by default ?
- Does the validator properly return 'self' ?
- Are the validation error messages clear and helpful?

Yes No

Exercise 2

Exercise 2 - Space Crew Management

Run the command: python3 ex2/space_crew.py

Verify that:

- CrewMember and SpaceMission models are properly defined
- The crew field accepts a list of CrewMember objects (nested models)
- Are the 5 ranks implemented ?
- Mission validation rules work (needs commander/captain, experience requirements, etc.). Te
- The demonstration shows mission creation with crew details
- Invalid missions (like no high-ranking officer) are properly rejected

If the exercise doesn't work as expected, tick "No."

Yes No

Nested Models Functionality - Exercise 2

Test the nested model functionality:

- Can you create individual CrewMember objects?
- Does the SpaceMission properly validate its crew list?
- Create a SpaceMission with one active Commander and one inactive Captain. What happen
- Are the mission validation rules logical and working?
- Try to create a SpaceMission without specifying the status. Is it "planned" by default ?
- Try to create a SpaceMission over budget. What happens?
- Does the code handle edge cases appropriately?

Yes No

Code Quality and Best Practices

Overall Code Quality

Review the overall code quality:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?
- Are type hints REQUIRED for all functions and methods (parameters and return values)?
- Docstrings are NOT required for this module.
- Is the code following Python naming conventions?
- Are the validation rules clearly implemented?
- Is the demonstration output clear and informative?

Yes No

Pydantic Usage

Check Pydantic usage:

- Are Field() constraints used appropriately?
- Is @model_validator used correctly for custom validation?
- Are enums used where appropriate?
- Does the code avoid deprecated Pydantic v1 features?

Yes No

Understanding and Learning

Assess the learner's understanding:

- Do the examples demonstrate proper Pydantic usage?
- Are the validation rules meaningful and well-thought-out?
- Does the code show progression from basic to advanced concepts?
- Would this code serve as a good learning example for others?

Yes No

Ratings

Don't forget to check the flag corresponding to the defense

Ok Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat

Concerning situation Leaks Forbidden function Can't support /

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33) Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2) Privacy policy (https://profile.intra.42.fr/legal/terms/5) General term of use of the site (https://profile.intra.42.fr/legal/terms/6) Rules of pro (https://profile.intra.42